Purpose

The goal of this assignment is to familiarize yourselves with process creation and file and signal handling, by writing a simple shell mysh (my simple shell) in a Linux environment and in the C (or C++) programming language.

Problem Statement

The shell must allow the user to create and handle new processes and give the user the opportunity to execute any system program that is already available.

Invoking the shell is simple and is done from the command line (prompt) as follows:

linux01.di.uoa.gr:> ./mysh
in-mysh-now:>

It is expected that mysh will use any information available in environment variables so that the commands are correctly interpreted by the operating system.

The commands entered into the shell must be parsed by the program and must not be invoked as they are. Implementations in which the shell commands are executed as-is (e.g. using system()) will not be accepted.

The Interface and Features of the Shell

mysh must provide:

- Redirection support
- Pipe support
- Execution of commands in the Background
- Signal handling
- Wild character support
- Alias management

a) Redirection Support

a.1) Input / output redirection (I/O redirection) (< , >):

Usage examples follow:

in-mysh-now:> myProgram > out.txt

The command myProgram runs and the result is stored in the file out.txt.

in-mysh-now:> myProgram < input.txt

The command myProgram runs and takes its input from input.txt.

in-mysh-now:> sort < file1 > file2

The contents of file1 are sorted and the result is stored in file2.

Note: The sort command is used as an example. Redirections must work for all commands of the underlying shell. The same applies to the rest of the examples that follow.

a.2) Append redirection to an existing file (>>)

The result of executing the left-hand part must be appended to an existing file e.g.

in-mysh-now:> cat file1 >> file2

b) Pipe Support

Provision of pipes and their combination with redirections, i.e.:

in-mysh-now:> cat file1 file2 file3 | sort > file4
the contents of the files file1 file2 file3 are used as input to the program sort and the result is written to the file file4.

c) Execution of Commands in the Background (&)

Ability to execute commands in the background using the & operator:

in-mysh-now:> sort file1 &; ls &;

When a command runs in the background it can and must run concurrently with the next command to be executed.

d) Wild Character Support

Ability to use wild characters. Wild characters can be used to specify a subset of the files in the current directory. For example the command:

in-mysh-now:> ls file.t?t*

must display the subset of the files in the current directory that have the prefix file and an extension that starts and ends with the character t and has any single character in between.

e) Alias Management

Ability to create / destroy aliases. For example:

in-mysh-now:> createalias myhome “cd /home/users/smith”;

creates an alias named myhome and if someone types it on the command line, it has an effect equivalent to executing the whole command (i.e. cd /home/users/smith).

Similarly there is also the command:

in-mysh-now:> destroyalias myhome;

which deactivates the alias.

f) Signal Handling

Your shell must be able to handle simple signals. More specifically the user must be able to

- send a control-C to the running process through the mysh prompt. The result must be the termination of the running program but NOT of the mysh shell.
- send a control-Z to the running process. The shell must also ignore the control-Z signal and forward it to the running program.

g) History Storage (myHistory)

Command history capability. The shell must be able to remember and recall in a simple way the user's last 20 commands. The user must be able to execute one of these commands without having to retype it.

Procedural Matters

- Your program must be written in C (or C++) and must run on the Department's Linux workstations. Code that does not compile there is considered not to compile.
- For additional announcements, follow the course forum on piazza.com. The full address is https://piazza.com/uoa.gr/spring2023/k24/home. Following the forum on Piazza is mandatory.
- Your code must consist of at least two (and preferably more) different files. The use of separate compilation is mandatory and your code must have a Makefile.
- Make sure that you follow good software engineering practices when implementing the assignment. The organization, readability and presence of comments in the code are part of your grade.
- Submission will be done through eclass.

- Your code must compile into the executable mysh exactly as the assignment specifies.

What to Submit

- All your work (source code, Makefile and README) in a tar.gz file named OnomaEponymoProject1.tar.gz (i.e. FirstnameLastnameProject1.tar.gz). Be careful to submit only code, Makefile and README and not the binaries. Your assignment will be compiled from scratch before it is graded.
- Any design choices you make, you must describe in a README (plain text file) that you will submit along with your code. The README needs to contain a brief and concise explanation of the choices you made in the design of your program. 1-2 pages of ASCII text are enough. Include the explanation and the instructions for compiling and running your program.
- The code you submit must be your own. The use of code not written by you or code written with the help of tools such as ChatGPT is prohibited.
- It is a good idea to keep a backup .tar of your assignment exactly as it was submitted on some easily accessible machine (department server, github, cloud).
- Correctly submitting a correct tar.gz containing your assignment's code and any files needed is solely your responsibility. Empty tar/tar.gz files or tar/tar.gz files that contain errors and cannot be extracted will not be graded.

What Will Be Graded

- The compliance of your code with the assignment's specifications.
- The organization and readability (along with the presence of comments) of the code.
- The use of a Makefile and separate compilation.

Other Important Remarks

- Assignments are individual.
- Anyone who submits / presents code that was not written by them receives a zero in the course.
- Although you are expected to discuss with friends and colleagues how you will attempt to solve the problem, copying code (of any form) is not allowed. Anyone found involved in code copying simply gets zero in the course. This applies to everyone involved regardless of who gave/took etc. We emphasize that you must take the appropriate measures so that your code is protected and is not stored somewhere accessible by other users (e.g., the excuse “I had put it in a github repo and they probably took it from there” is not acceptable).
- Programming assignments may be submitted at most 3 days late with a 5% penalty for each day of delay. Beyond these 3 days, assignments cannot be submitted.
- Your program must be written in C or C++. You may only use commands that are available on the Department's Linux machines. A program that may compile or run on your personal computer but not on the Department's Linux machines is considered not to compile or run respectively.